# Unidad 3 — React Core

En esta unidad damos el salto desde el JavaScript moderno que ya dominas (Unidades 1 y 2) hacia **React**, la tecnología más extendida hoy para construir interfaces de usuario en la web. Aprenderás a pensar en **componentes**, a describir tu interfaz de forma **declarativa** y a gestionar el **estado** que hace que tus aplicaciones cobren vida. Trabajaremos con **React 19.2**, la versión estable en junio de 2026, y siempre con **componentes como funciones** (los componentes de clase son hoy legado). Esta es una base sólida sobre la que construiremos en las unidades siguientes (formularios y Actions en la U4, Next.js en la U6).

## Índice

1. ¿Qué es React?
2. Puesta en marcha
3. JSX
4. Componentes
5. Props
6. Renderizado condicional y listas
7. Estado con `useState`
8. Eventos
9. `useEffect`
10. Otros hooks esenciales y hooks personalizados
11. El hook `use` y los hooks de Actions (React 19)
12. Levantar el estado y flujo unidireccional
13. Pensar en React
14. Buenas prácticas y errores comunes
15. Ejercicios propuestos
16. Recursos y referencias

## ¿Qué es React?

React es una **librería de JavaScript para construir interfaces de usuario**. Fíjate en la palabra *librería*: React no te impone una estructura de carpetas, ni un router, ni una forma de

obtener datos del servidor. Se centra en una sola cosa: ayudarte a describir **qué** debe aparecer en pantalla en función de los datos de tu aplicación.

Esa es la idea clave del **enfoque declarativo**. En JavaScript "a mano" (lo que se llama estilo *imperativo*) tú das órdenes paso a paso: "crea este `<li>`, añádelo a esta lista, ahora cambia el texto de aquel `<span>`...". En React, en cambio, **declaras** cómo debe verse la interfaz para un estado dado, y React se encarga de aplicar los cambios necesarios en el DOM cuando ese estado cambia.

```
// Imperativo (sin React): tú manipulas el DOM directamente
const titulo = document.createElement("h1");
titulo.textContent = "Hola";
document.body.appendChild(titulo);

// Declarativo (con React): describes el resultado
function Saludo() {
  return <h1>Hola</h1>;
}
```

React está **basado en componentes**: tu interfaz se divide en piezas reutilizables (un botón, una tarjeta, un formulario, una página entera) que se combinan como las piezas de un Lego.

## El DOM virtual (idea general)

Manipular el DOM real es costoso. React mantiene en memoria una representación ligera de tu interfaz llamada **DOM virtual**. Cuando el estado cambia, React genera una nueva versión del DOM virtual, la **compara** con la anterior (un proceso llamado *reconciliación*) y calcula el conjunto mínimo de cambios reales que hay que aplicar al DOM del navegador. Tú no tienes que pensar en ello: solo describes el resultado y React optimiza el "cómo".

> 📌 Nota: no necesitas dominar los detalles internos del DOM virtual para usar React con soltura. Basta con entender que React actualiza el DOM real de forma eficiente por ti.

## ¿Por qué se usa con un framework?

React por sí solo no resuelve cosas como el enrutado entre páginas, la obtención de datos en el servidor o la optimización para SEO. Por eso, en proyectos reales, React suele usarse dentro de un **framework** que aporta esas piezas. El más popular es **Next.js**, que estudiaremos a fondo en la **Unidad 6**. La propia documentación oficial (react.dev)

recomienda empezar nuevos proyectos con un framework. En esta unidad, para centrarnos en los fundamentos de React, usaremos una **SPA** (*Single Page Application*) sencilla creada con Vite.

## Puesta en marcha

> ⚠️ Cuidado: **Create React App está descontinuado**. No lo uses. La herramienta estándar hoy para una SPA con React es **Vite**, y para aplicaciones completas, un framework como Next.js.

Para crear un proyecto React con Vite:

```
npm create vite@latest mi-app -- --template react
cd mi-app
npm install
npm run dev
```

Esto levanta un servidor de desarrollo (normalmente en `http://localhost:5173`) con recarga instantánea. La estructura típica de un proyecto es:

```
mi-app/
├── index.html          # punto de entrada HTML
├── package.json        # dependencias y scripts
├── vite.config.js      # configuración de Vite
└── src/
    ├── main.jsx        # arranca React y monta la app en el DOM
    ├── App.jsx         # componente raíz
    └── components/     # tus componentes (por convención)
```

El fichero `main.jsx` conecta React con el HTML:

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

> 💡 Consejo: `StrictMode` no afecta a producción; en desarrollo activa comprobaciones extra que te ayudan a detectar errores (por ejemplo, ejecuta dos veces ciertas funciones para que descubras efectos secundarios mal escritos).

> 💡 Consejo: instala la extensión **React Developer Tools** (disponible para Chrome, Firefox y Edge). Añade al inspector del navegador pestañas para explorar el árbol de componentes y ver las props y el estado de cada uno en tiempo real: es imprescindible para depurar aplicaciones React.

## JSX

**JSX** es una extensión de la sintaxis de JavaScript que te permite escribir algo parecido a HTML dentro de tu código. No es HTML ni una cadena de texto: es **azúcar sintáctico** que se transforma en llamadas a funciones de React. Te deja describir la interfaz de forma legible.

```jsx
const elemento = <h1 className="titulo">¡Hola, mundo!</h1>;
```

### Expresiones con llaves `{}`

Dentro del JSX puedes incrustar cualquier **expresión** de JavaScript usando llaves:

```jsx
const nombre = "Ada";
const elemento = <p>Bienvenida, {nombre}. Hoy es {new Date().getFullYear()}.</p>;
```

Recuerda: dentro de `{}` van **expresiones** (algo que produce un valor), no sentencias. No puedes poner un `if` o un `for` ahí dentro, pero sí un operador ternario o un `.map()`.

## Diferencias con HTML

Como JSX es JavaScript, algunos nombres de atributos cambian porque chocan con palabras reservadas:

| HTML | JSX | Motivo |
|---|---|---|
| `class` | `className` | `class` es palabra reservada en JS |
| `for` | `htmlFor` | `for` es palabra reservada en JS |
| `onclick` | `onClick` | eventos en *camelCase* |
| `tabindex` | `tabIndex` | atributos en *camelCase* |

Además, todas las etiquetas deben **cerrarse**: `<img />`, `<br />`, `<input />`.

## Un solo nodo raíz y los Fragments

Un componente debe devolver **un único nodo raíz**. Si necesitas devolver varios elementos hermanos sin añadir un `<div>` extra al DOM, usa un **Fragment**, que se escribe con la sintaxis vacía `<>...</>`:

```
function Datos() {
  return (
    <>
      <h2>Título</h2>
      <p>Párrafo asociado.</p>
    </>
  );
}
```

## Comentarios

Dentro del JSX, los comentarios van como expresiones de JavaScript:

```jsx
function Ejemplo() {
  return (
    <div>
      {/* Esto es un comentario en JSX */}
      <p>Contenido</p>
    </div>
  );
}
```

## Componentes

Un **componente** es una **función de JavaScript que devuelve JSX**. Su nombre se escribe en **PascalCase** (primera letra de cada palabra en mayúscula), y la mayúscula inicial no es una simple convención: es una **regla de JSX**. Una etiqueta que empieza por minúscula ( `<button>` , `<div>` ) se interpreta como una etiqueta del DOM, mientras que una que empieza por mayúscula ( `<Boton>` ) se interpreta como un componente.

```jsx
function Boton() {
  return <button>Pulsa aquí</button>;
}
```

### Composición

Los componentes se **componen** unos dentro de otros, igual que las funciones se llaman entre sí:

```jsx
function Tarjeta() {
  return (
    <article className="tarjeta">
      <h3>Producto</h3>
      <Boton />
    </article>
  );
}
```

### Exportar e importar

Cada componente suele vivir en su propio fichero y se exporta para reutilizarlo:

```jsx
// Boton.jsx
export default function Boton() {
  return <button>Pulsa aquí</button>;
}
```

```jsx
// App.jsx
import Boton from "./Boton.jsx";

export default function App() {
  return <Boton />;
}
```

> 📌 Nota: los **componentes de clase** ( `class X extends React.Component` ) son la forma **antigua** de escribir componentes. Hoy se usan siempre funciones con hooks. Los mencionamos solo para que los reconozcas en código heredado.

## Props

Las **props** (de *properties*) son los datos que un componente padre pasa a un componente hijo. Se escriben como atributos en el JSX y el hijo los recibe como un objeto:

```jsx
function Saludo(props) {
  return <p>Hola, {props.nombre}</p>;
}

// Uso:
<Saludo nombre="Ada" />
```

### Desestructuración

Lo habitual es **desestructurar** las props directamente en los parámetros de la función para un código más limpio:

```jsx
function Saludo({ nombre }) {
  return <p>Hola, {nombre}</p>;
}
```

### Valores por defecto

Como son parámetros de función normales, puedes darles valores por defecto con la sintaxis de JavaScript:

```
function Boton({ texto = "Aceptar" }) {
  return <button>{texto}</button>;
}
```

### La prop `children`

`children` es una prop especial: representa el contenido que se escribe **entre** las etiquetas de apertura y cierre del componente. Permite crear componentes "envoltorio":

```
function Panel({ titulo, children }) {
  return (
    <section className="panel">
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}

// Uso:
<Panel titulo="Avisos">
  <p>Este párrafo llega como children.</p>
</Panel>
```

### Las props son de solo lectura

Regla fundamental: **un componente nunca debe modificar sus propias props**. Las props son **inmutables** desde el punto de vista del hijo; fluyen siempre del padre hacia el hijo. Si un componente necesita cambiar un valor a lo largo del tiempo, eso es **estado**, no props.

> ⚠️ Cuidado: nunca hagas `props.nombre = "otro"`. Las props son de solo lectura. Si lo intentas, romperás el flujo de datos de React.

# Renderizado condicional y listas

## Renderizado condicional

Puedes decidir qué mostrar según una condición. Hay tres patrones habituales.

**1. Con `if` antes del `return`** (cuando devuelves estructuras muy distintas):

```
function Saludo({ usuario }) {
  if (!usuario) {
    return <p>Inicia sesión, por favor.</p>;
  }
  return <p>Hola, {usuario.nombre}</p>;
}
```

**2. Con el operador ternario** (dentro del JSX, para elegir entre dos opciones):

```
function Estado({ activo }) {
  return <span>{activo ? "En línea" : "Desconectado"}</span>;
}
```

**3. Con el operador `&&`** (para mostrar algo solo si la condición es verdadera):

```
function Bandeja({ mensajes }) {
  return (
    <div>
      {mensajes.length > 0 && <p>Tienes {mensajes.length} mensajes nuevos.</p>}
    </div>
  );
}
```

> ⚠️ Cuidado con el `&&` y los números: `{mensajes.length && <p>...</p>}` renderizaría un `0` en pantalla si la lista está vacía, porque `0` es *falsy* pero React lo muestra. Usa siempre una comparación que devuelva un booleano: `mensajes.length > 0 && ...`.

### Renderizado de listas

Para mostrar una lista de elementos usamos el método `.map()` de los arrays, que ya conoces:

```
function ListaTareas({ tareas }) {
  return (
    <ul>
      {tareas.map((tarea) => (
        <li key={tarea.id}>{tarea.texto}</li>
      ))}
    </ul>
  );
}
```

#### La prop `key` y por qué importa

Cada elemento de una lista necesita una prop `key` única y estable entre sus hermanos. La `key` permite a React identificar qué elementos han cambiado, se han añadido o eliminado durante la reconciliación, y así actualizar el DOM correctamente.

> ⚠️ Cuidado: **no uses el índice del array como `key`** si la lista puede reordenarse, filtrarse o crecer por el medio. Usa un identificador estable (como un `id` de base de datos). Usar el índice en listas dinámicas provoca errores sutiles, como inputs que conservan el valor equivocado al reordenar.

## Estado con `useState`

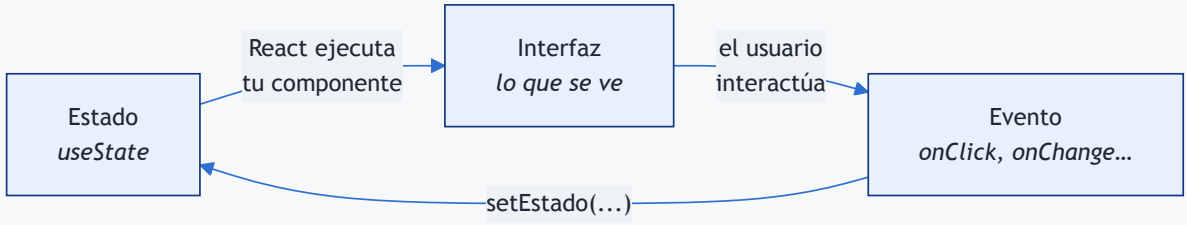
El **estado** es la "memoria" de un componente: datos que cambian con el tiempo y que, al cambiar, deben provocar que la interfaz se actualice. Una variable normal no sirve, porque React no se entera de que ha cambiado y, además, su valor se perdería en cada renderizado. Para eso existe el hook `useState`.

```jsx
import { useState } from "react";

function Contador() {
  const [cuenta, setCuenta] = useState(0);

  return (
    <button onClick={() => setCuenta(cuenta + 1)}>
      Has pulsado {cuenta} veces
    </button>
  );
}
```

`useState(valorInicial)` devuelve un array de dos posiciones: el **valor actual** del estado y una **función para actualizarlo**. Cuando llamas a la función actualizadora ( `setCuenta` ), React vuelve a ejecutar el componente (un **re-render**) con el nuevo valor.




Este es **el ciclo entero de React**, y merece la pena grabárselo: tú no tocas la pantalla, tú cambias el estado. React se encarga del resto. Cuando algo no se actualiza en tu aplicación, la pregunta siempre es la misma: ¿de verdad ha cambiado el estado?

## El estado es una instantánea (snapshot)

Esta es una de las ideas que más cuesta interiorizar. Dentro de un render concreto, el valor del estado es **fijo**: es una *instantánea* tomada en el momento de renderizar. Mira este ejemplo:

```
function Ejemplo() {
  const [cuenta, setCuenta] = useState(0);

  function manejar() {
    setCuenta(cuenta + 1);
    setCuenta(cuenta + 1);
    setCuenta(cuenta + 1);
  }

  return <button onClick={manejar}>Cuenta: {cuenta}</button>;
}
```

Aunque llamamos tres veces a `setCuenta`, la cuenta solo sube **1**, no 3. ¿Por qué? Porque en ese render `cuenta` vale `0`, así que las tres líneas son `setCuenta(0 + 1)`, es decir, `setCuenta(1)` tres veces.

El ejemplo siguiente **no es React**: es una simulación en JavaScript puro de lo que hace React por dentro, para que veas que aquí no hay magia, solo una variable capturada por un *closure* (Unidad 2) que no cambia durante el render.

```
// Simulación mínima de useState: la cola de actualizaciones y el valor del render
let valorReal = 0;
const cola = [];

function render() {
  const cuenta = valorReal;                 // ← la INSTANTÁNEA de este render
  const setCuenta = (v) => cola.push(v);    // encola, no cambia 'cuenta' aquí

  // --- esto es lo que escribes tú en el componente ---
  setCuenta(cuenta + 1);
  setCuenta(cuenta + 1);
  setCuenta(cuenta + 1);
  // ---------------------------------------------------

  console.log("Durante el render, cuenta vale:", cuenta);
  console.log("Se ha encolado:", cola.slice());

  // React procesa la cola y prepara el siguiente render
  for (const v of cola) valorReal = typeof v === "function" ? v(valorReal) : v;
  cola.length = 0;
  console.log("→ En el siguiente render valdrá:", valorReal);
}

render();

// Ahora prueba a cambiar las tres líneas por la forma con función:
//    setCuenta(c => c + 1);
// y vuelve a ejecutar. ¿Por qué ahora sí sube de 3 en 3?
```

## La función actualizadora

Para basar la actualización en el valor **anterior real**, pasa una función a la función actualizadora:

```
function manejar() {
  setCuenta((c) => c + 1);
  setCuenta((c) => c + 1);
  setCuenta((c) => c + 1);
}
// Ahora sí sube de 3 en 3
```

## Actualizaciones por lotes (batching)

React **agrupa** (*batches*) varias actualizaciones de estado que ocurren dentro del mismo manejador de evento y hace **un solo re-render** al final, por eficiencia. Desde React 18, este *automatic batching* agrupa también las actualizaciones que ocurren en código **asíncrono**: dentro de promesas, callbacks de `setTimeout` o manejadores de eventos nativos. Por eso el estado no cambia "en mitad" de tu función: cambia para el **siguiente** render.

## Actualizar objetos y arrays de forma inmutable

Nunca modifiques el estado directamente. React detecta los cambios comparando referencias, así que debes crear **nuevos** objetos o arrays en lugar de mutar los existentes.

Esta es **la razón exacta** por la que a veces "cambias el estado y no se repinta". React compara con `Object.is` el valor anterior y el nuevo; si son el mismo objeto, para él no ha pasado nada:

```
const usuario = { nombre: "Ana", edad: 29 };

// ❌ Mutar: cambias el contenido, pero la referencia es la misma
const mutado = usuario;
mutado.edad = 30;
console.log("¿React ve un cambio?", !Object.is(usuario, mutado)); // false → NO rep

// ✅ Copiar: objeto nuevo, referencia nueva
const copiado = { ...usuario, edad: 31 };
console.log("¿React ve un cambio?", !Object.is(usuario, copiado)); // true → repint

// Lo mismo con arrays: push muta, los métodos que devuelven copia no
const tareas = [{ id: 1, hecha: false }];
const empujado = tareas; empujado.push({ id: 2, hecha: false });
const concatenado = [...tareas, { id: 3, hecha: false }];
console.log("push  →", !Object.is(tareas, empujado));      // false
console.log("spread →", !Object.is(tareas, concatenado));  // true
```

```
// ❌ MAL: mutar directamente
usuario.edad = 30;
setUsuario(usuario);

// ✅ BIEN: crear un objeto nuevo con spread
setUsuario({ ...usuario, edad: 30 });
```

```
// Añadir a un array sin mutar
setTareas([...tareas, nuevaTarea]);

// Eliminar de un array sin mutar
setTareas(tareas.filter((t) => t.id !== id));

// Modificar un elemento sin mutar
setTareas(tareas.map((t) => (t.id === id ? { ...t, hecha: true } : t)));
```

## Eventos

En React los eventos se asignan con atributos en *camelCase* (`onClick`, `onChange`, `onSubmit`...) y reciben una **función manejadora**:

```
function Boton() {
  function manejarClick() {
    alert("¡Clic!");
  }
  return <button onClick={manejarClick}>Pulsa</button>;
}
```

> ⚠️ Cuidado: se pasa la función **sin paréntesis** (`onClick={manejarClick}`). Si pones `onClick={manejarClick()}` la estarías **ejecutando** durante el render, no al hacer clic.

### El evento sintético

React envuelve los eventos nativos del navegador en un objeto unificado llamado **evento sintético** (*SyntheticEvent*), que se comporta igual en todos los navegadores. Tiene los métodos habituales como `preventDefault()` o `stopPropagation()`:

```
function Formulario() {
  function manejarEnvio(evento) {
    evento.preventDefault(); // evita que la página se recargue
    console.log("Formulario enviado");
  }
  return <form onSubmit={manejarEnvio}>{/* ... */}</form>;
}
```

### Pasar argumentos a un manejador

Para pasarle datos, envuelve la llamada en una función flecha:

```
function Lista({ items }) {
  function borrar(id) {
    console.log("Borrar", id);
  }
  return (
    <ul>
      {items.map((item) => (
        <li key={item.id} onClick={() => borrar(item.id)}>
          {item.texto}
        </li>
      ))}
    </ul>
  );
}
```

## `useEffect`

Un **efecto** (`useEffect`) te permite sincronizar tu componente con **sistemas externos** a React: llamadas a una API, suscripciones, temporizadores, el `localStorage`, etc. El código de un efecto se ejecuta **después** de que React haya pintado en pantalla.

```
import { useState, useEffect } from "react";

function Reloj() {
  const [hora, setHora] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setHora(new Date()), 1000);
    // Función de limpieza: se ejecuta al desmontar o antes de re-ejecutar
    return () => clearInterval(id);
  }, []); // array de dependencias vacío: se ejecuta una sola vez al montar

  return <p>{hora.toLocaleTimeString()}</p>;
}
```

> ⚠️ Cuidado: con `<StrictMode>` (que activamos en la puesta en marcha), React en **desarrollo** monta el componente, ejecuta la limpieza y vuelve a montar el efecto (monta → limpia → remonta). Por eso verás el efecto ejecutarse **dos veces** en la consola: es intencionado (sirve para comprobar que tu función de limpieza funciona bien) y **solo ocurre en desarrollo**, nunca en producción.

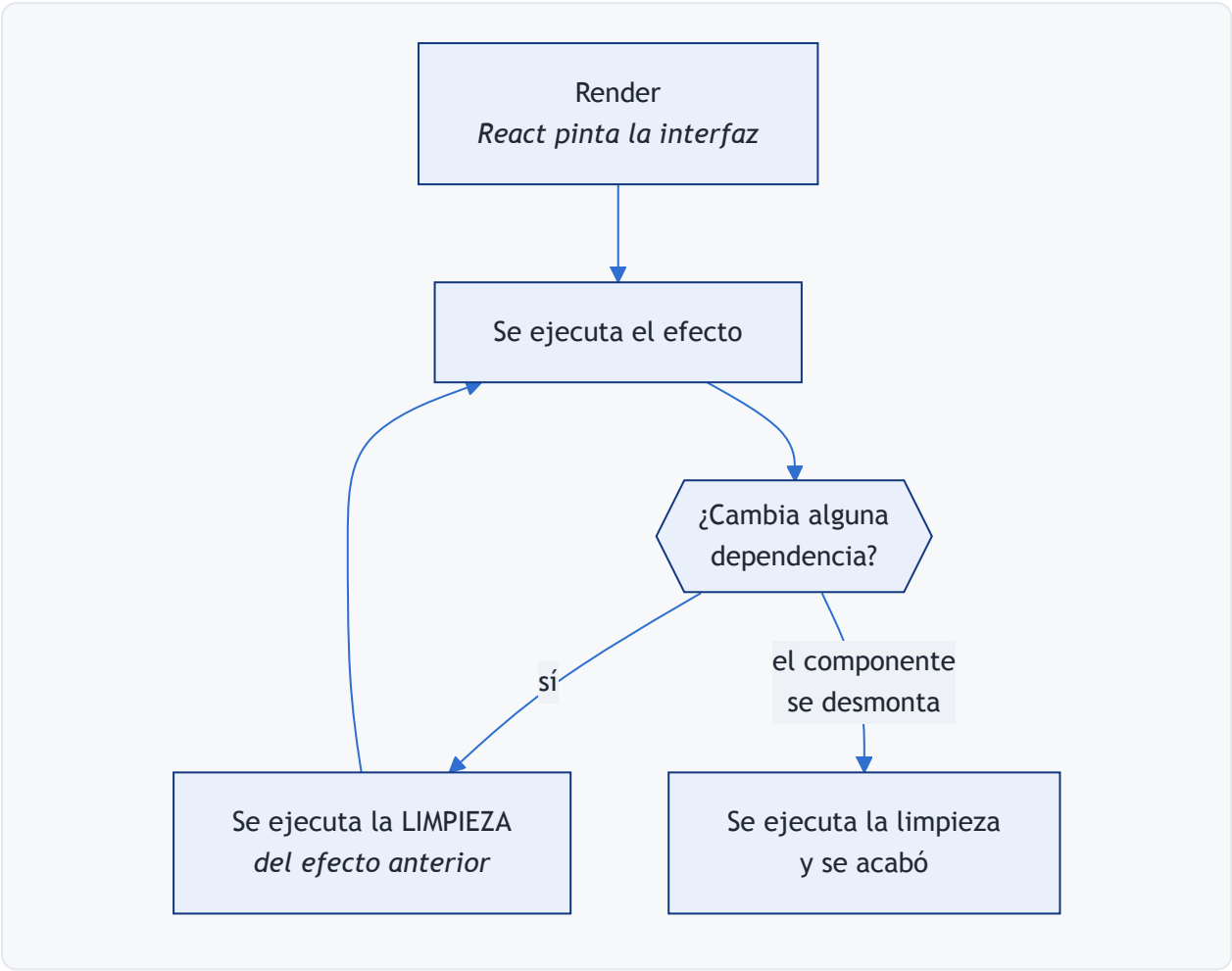
## El array de dependencias

El segundo argumento controla **cuándo** se vuelve a ejecutar el efecto:

| Forma | Cuándo se ejecuta |
|---|---|
| Sin segundo argumento | Después de **cada** render |
| `[]` (vacío) | Solo una vez, al **montar** |
| `[a, b]` | Al montar y cada vez que `a` o `b` cambian |

## La función de limpieza (cleanup)

Si tu efecto crea algo que debe deshacerse (un temporizador, una suscripción, un *listener*), devuelve una función de limpieza. React la ejecuta antes de volver a correr el efecto y cuando el componente se **desmonta**. Esto evita fugas de memoria y comportamientos duplicados.




Fíjate en que la limpieza no solo corre al desmontar: corre **antes de cada nueva ejecución** del efecto. Si tu efecto abre un temporizador y no lo cierras, cada cambio de dependencia deja uno más corriendo por detrás.

## Cuándo NO usar un efecto

Un error muy común de quien empieza es abusar de `useEffect`. La guía oficial **"You Might Not Need an Effect"** (react.dev) explica que muchos efectos sobran. En particular:

- **No** uses un efecto para **transformar datos para el render**: si puedes calcular un valor a partir de las props o el estado, hazlo directamente durante el render (o con `useMemo` si es costoso).
- **No** uses un efecto para responder a una **interacción del usuario**: esa lógica va en el **manejador del evento**.

```
// ❌ Innecesario: un efecto para derivar un valor
const [nombreCompleto, setNombreCompleto] = useState("");
useEffect(() => {
  setNombreCompleto(nombre + " " + apellido);
}, [nombre, apellido]);

// ✅ Calcúlalo directamente durante el render
const nombreCompleto = nombre + " " + apellido;
```

### Reglas de los hooks

Los hooks (toda función que empieza por `use`) tienen dos reglas innegociables:

1. **Solo se llaman en el nivel superior** del componente. Nunca dentro de bucles, condicionales o funciones anidadas. React necesita que se llamen siempre en el mismo orden en cada render.
2. **Solo se llaman desde componentes de React** o desde otros hooks personalizados.

> 💡 Consejo: el plugin ESLint de React (`eslint-plugin-react-hooks`) verifica automáticamente estas reglas y te avisa de dependencias que faltan en tus efectos. Actívalo siempre.

## Otros hooks esenciales y hooks personalizados

### `useRef`

`useRef` crea una "caja" mutable cuyo valor persiste entre renders **sin** provocar re-renders al cambiar. Tiene dos usos principales:

**1. Acceder a un nodo del DOM:**

```
function CampoTexto() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Enfocar</button>
    </>
  );
}
```

**2. Guardar un valor mutable** que no debe disparar render (por ejemplo, el `id` de un `setInterval`).

> 📌 Nota (React 19): ahora `ref` se puede pasar como una **prop normal** a tus componentes de función. En la mayoría de casos **ya no necesitas** `forwardRef`, que era la forma antigua de reenviar refs.

## `useContext`

Cuando un dato debe llegar a componentes muy anidados, pasarlo prop a prop por cada nivel intermedio se vuelve tedioso: es el problema del **prop drilling**. `useContext` permite compartir un valor con todo un subárbol sin pasarlo manualmente.

```
import { createContext, useContext } from "react";

const TemaContext = createContext("claro");

function App() {
  return (
    <TemaContext value="oscuro">
      <Barra />
    </TemaContext>
  );
}

function Barra() {
  const tema = useContext(TemaContext);
  return <div className={`barra ${tema}`}>Barra en tema {tema}</div>;
}
```

> 📌 Nota (React 19): puedes usar `<TemaContext>` directamente como proveedor, sin escribir `<TemaContext.Provider>`.

### `useReducer`

Para estado **complejo** —con varias propiedades relacionadas o muchas transiciones— `useReducer` centraliza la lógica de actualización en una única función llamada *reducer*. Es una alternativa a `useState` cuando este se queda corto.

```
import { useReducer } from "react";

function reducer(estado, accion) {
  switch (accion.tipo) {
    case "incrementar":
      return { cuenta: estado.cuenta + 1 };
    case "reiniciar":
      return { cuenta: 0 };
    default:
      return estado;
  }
}

function Contador() {
  const [estado, dispatch] = useReducer(reducer, { cuenta: 0 });
  return (
    <>
      <p>{estado.cuenta}</p>
      <button onClick={() => dispatch({ tipo: "incrementar" })}>+1</button>
      <button onClick={() => dispatch({ tipo: "reiniciar" })}>Reiniciar</button>
    </>
  );
}
```

### `useMemo` y `useCallback` (memoización)

Estos dos hooks sirven para **memoizar** (guardar en caché) resultados entre renders y evitar trabajo innecesario:

- `useMemo` memoiza el **resultado** de un cálculo costoso.
- `useCallback` memoiza una **función**, útil cuando se pasa a componentes hijos optimizados para que no se vuelvan a renderizar sin necesidad.

```
const valorCalculado = useMemo(() => calculoCostoso(datos), [datos]);
const manejar = useCallback(() => hacerAlgo(id), [id]);
```

> 💡 Consejo: **no abuses** de la memoización. Solo aporta cuando hay un cálculo realmente costoso o un problema de rendimiento medido. Además, el **React Compiler** —que alcanzó su versión estable 1.0 en octubre de 2025— optimiza automáticamente los re-renders, por lo que escribir `useMemo` y `useCallback` a mano es cada vez menos necesario. La tendencia es escribir menos memoización manual.

## Hooks personalizados

Cuando una misma lógica con hooks se repite en varios componentes, puedes extraerla a un **hook personalizado**: una función normal de JavaScript cuyo nombre empieza por `use` y que llama a otros hooks en su interior. No es una API especial de React; es simplemente una forma de **reutilizar lógica con estado** entre componentes.

Por ejemplo, el reloj que escribimos en la sección de `useEffect` (estado con la hora + `setInterval` + limpieza) puede extraerse a un hook `useReloj`:

```
import { useState, useEffect } from "react";

function useReloj() {
  const [hora, setHora] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setHora(new Date()), 1000);
    return () => clearInterval(id);
  }, []);

  return hora;
}
```

Ahora cualquier componente puede consumirlo con una sola línea, sin repetir el efecto ni la limpieza:

```
function Reloj() {
  const hora = useReloj();
  return <p>{hora.toLocaleTimeString()}</p>;
}

function BarraEstado() {
  const hora = useReloj();
  return <footer>Última actualización: {hora.toLocaleTimeString()}</footer>;
}
```

Fíjate en dos detalles importantes:

- Los hooks personalizados comparten **lógica**, no estado: cada componente que llama a `useReloj()` obtiene su **propio** estado independiente, igual que si escribiera el `useState` y el `useEffect` dentro.
- El prefijo `use` no es decorativo: conecta con las **reglas de los hooks**. Al empezar por `use`, React (y el plugin de ESLint) saben que dentro puede haber hooks, así que se le aplican las mismas reglas: llamarlo solo en el nivel superior y solo desde componentes u otros hooks. De hecho, la segunda regla ("solo desde componentes de React o desde otros hooks personalizados") existe precisamente para que este patrón funcione.

> 💡 Consejo: si te descubres copiando y pegando un `useEffect` con su limpieza entre componentes, es una señal clara de que ahí hay un hook personalizado esperando a ser extraído.

## El hook `use` y los hooks de Actions (React 19)

React 19 introdujo varias APIs que aplicaremos sobre todo en la **Unidad 4** (formularios y datos). Las presentamos aquí brevemente para que las conozcas.

### El hook `use`

`use` permite **leer el valor de una promesa** (o de un contexto) durante el render, integrándose con `Suspense` para mostrar un estado de carga mientras los datos llegan. A diferencia de otros hooks, `use` **sí** puede llamarse condicionalmente.

```jsx
import { use } from "react";

function Perfil({ usuarioPromesa }) {
  const usuario = use(usuarioPromesa); // espera a que la promesa resuelva
  return <h1>{usuario.nombre}</h1>;
}
```

> ⚠️ Cuidado: la promesa **no debe crearse durante el render**: debe llegar cacheada, creada por un componente padre (como en el ejemplo, donde viene por props) o proporcionada por un framework. Escribir `use(fetch(...))` dentro del componente es un error típico: crearía una promesa **nueva en cada render**. Además, el componente que llama a `use(promesa)` necesita un límite `<Suspense>` **por encima** en el árbol que muestre el estado de carga mientras la promesa se resuelve.

### Hooks de Actions

Las **Actions** simplifican el manejo de operaciones asíncronas (como enviar un formulario), gestionando automáticamente estados de pendiente, errores y actualizaciones. Los nuevos hooks son:

- `useActionState`: gestiona el estado resultante de una acción (datos, error, si está pendiente).
- `useFormStatus`: da información sobre el estado de envío del `<form>` padre (por ejemplo, `pending`).
- `useOptimistic`: muestra un estado **optimista** (un cambio provisional en la interfaz) mientras la operación real termina.

No te preocupes por dominarlos ahora: los trabajaremos en detalle en la **Unidad 4**.

## Levantar el estado y flujo unidireccional

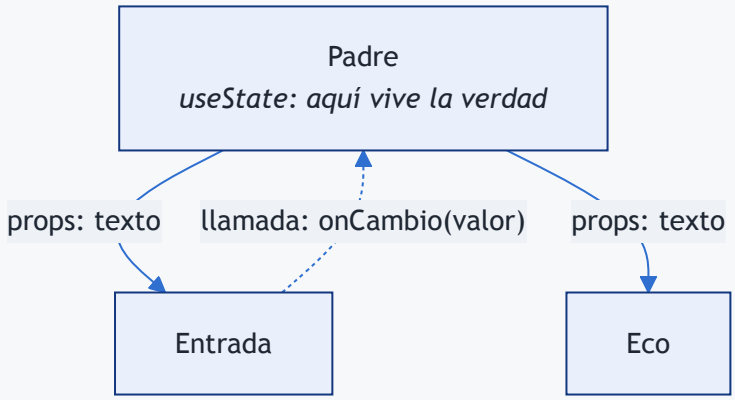
En React los datos fluyen **en una sola dirección**: de **padres a hijos** a través de las props. Esto se llama **flujo de datos unidireccional** y hace que las aplicaciones sean predecibles: si algo se ve mal en pantalla, sabes que el origen está "arriba".

Pero ¿qué pasa cuando dos componentes hermanos necesitan compartir el mismo estado? La técnica es **levantar el estado** (*lifting state up*): se mueve el estado al **ancestro común más cercano**, que lo pasa hacia abajo como prop y entrega también una función para modificarlo.

```jsx
function Padre() {
  const [texto, setTexto] = useState("");
  return (
    <>
      <Entrada texto={texto} onCambio={setTexto} />
      <Eco texto={texto} />
    </>
  );
}

function Entrada({ texto, onCambio }) {
  return <input value={texto} onChange={(e) => onCambio(e.target.value)} />;
}

function Eco({ texto }) {
  return <p>Escribiste: {texto}</p>;
}
```




Los datos **bajan** por props (flechas continuas) y los cambios **suben** como llamadas a funciones (flecha de puntos). No hay ninguna flecha entre `Entrada` y `Eco`: los hermanos nunca se hablan directamente, se coordinan a través del padre. Esa es toda la idea del flujo unidireccional.

Aquí el estado vive en `Padre`. `Entrada` lo modifica llamando a la función `onCambio` que recibe por props (los datos *suben* mediante llamadas a funciones), y tanto `Entrada` como `Eco` lo leen (los datos *bajan* por props). El padre es la **única fuente de verdad**. Fíjate además en el patrón `value` + `onChange` del `<input>`: esto se conoce como *input controlado* (el valor del campo lo dicta el estado de React), y lo desarrollaremos en detalle en la **Unidad 4**.

## Pensar en React

El sitio oficial propone un método para construir cualquier interfaz. Resumido:

1. **Descompón la UI en componentes.** Mira el diseño y dibuja recuadros alrededor de cada pieza. Una buena regla es el **principio de responsabilidad única**: un componente debería hacer una sola cosa.
2. **Organízalos en una jerarquía** (qué componente contiene a cuáles).
3. **Construye una versión estática** primero, solo con props, sin estado, que reciba los datos y los muestre.
4. **Identifica el estado mínimo necesario.** Pregúntate de cada dato: ¿cambia con el tiempo? ¿se puede calcular a partir de otro? Si se puede calcular, **no** es estado.
5. **Decide dónde vive cada estado** (recuerda: en el ancestro común si lo comparten varios).
6. **Añade el flujo inverso** (las funciones que permiten a los hijos comunicar cambios al padre).

> 💡 Consejo: un error típico es crear estado redundante. Si un valor se puede **derivar** de las props o de otro estado, calcúlalo durante el render en lugar de guardarlo aparte. Menos estado = menos errores.

## Buenas prácticas y errores comunes

- **Componentes pequeños y con un solo propósito.** Si un componente crece demasiado, divídelo.
- **Nunca mutes el estado ni las props.** Crea siempre copias nuevas (`{...obj}`, `[...arr]`).
- `key` **estable en las listas.** Evita el índice del array salvo en listas estáticas.
- **No llames a** `setState` **durante el render.** Provoca un bucle infinito. Hazlo en manejadores de eventos o efectos.
- **No olvides la limpieza en** `useEffect` cuando creas temporizadores o suscripciones.
- **No abuses de** `useEffect`**.** Pregúntate si el cálculo puede hacerse durante el render o si la lógica pertenece a un manejador de evento.
- **Respeta las reglas de los hooks:** nivel superior, nunca dentro de condicionales o bucles.
- **Pasa funciones, no llamadas, a los manejadores:** `onClick={f}`, no `onClick={f()}`.
- **Cuidado con** `&&` **y valores numéricos:** convierte a booleano para no pintar un `0`.

| Error común | Solución |
|---|---|
| `props.x = ...` | Usar estado; las props son inmutables |
| `setLista(lista.push(x))` | `setLista([...lista, x])` |
| `key={index}` en lista dinámica | `key={item.id}` |
| `onClick={borrar()}` | `onClick={() => borrar()}` |
| Efecto para derivar datos | Calcular durante el render |
| Hook dentro de un `if` | Hook en el nivel superior |

## Ejercicios propuestos

1. **Hola componente.** Crea un proyecto con Vite y un componente `Saludo` que reciba una prop `nombre` y muestre "Hola, {nombre}". Renderízalo tres veces con nombres distintos desde `App`.
2. **Tarjeta de perfil.** Crea un componente `Tarjeta` que reciba props (`nombre`, `rol`, `online`) y use `children` para una descripción. Aplica renderizado condicional con `&&` para mostrar un punto verde "En línea" solo si `online` es `true`.
3. **Contador con función actualizadora.** Implementa un contador con `useState`. Añade un botón "+3" que llame a `setCuenta` tres veces. Hazlo funcionar correctamente usando la función actualizadora `(c) => c + 1` y explica en un comentario por qué con `cuenta + 1` no funcionaría.
4. **Lista de tareas (inmutabilidad).** Crea una lista de tareas con un input para añadir, un botón para marcar como hecha y otro para borrar. Gestiona el array de estado **sin mutarlo** (usa spread, `map` y `filter`) y asigna `key` estable a cada `<li>`.
5. **Filtro derivado (sin estado redundante).** Partiendo del ejercicio 4, añade un campo de búsqueda. La lista filtrada **no** debe guardarse en estado: calcúlala durante el render a partir del texto de búsqueda y el array de tareas.
6. **Reloj con `useEffect`.** Crea un componente `Reloj` que muestre la hora actual y se actualice cada segundo con `setInterval`. Incluye la **función de limpieza** correcta. Añade un botón en el padre para montar/desmontar el reloj y comprueba en la consola que el intervalo se limpia al desmontar.
7. **Levantar el estado.** Crea dos componentes hermanos: uno con un campo de texto y otro que muestre en mayúsculas lo que se escriba. Levanta el estado al padre común y pásalo

por props en ambos sentidos.

8. **Tema con `useContext`.** Implementa un selector de tema (claro/oscuro) usando `createContext` y `useContext`, de modo que un componente nieto lea el tema sin recibirlo por props intermedias (evita el *prop drilling*).
9. **Reducer de carrito.** Reescribe la lógica de un pequeño carrito de la compra (añadir producto, quitar producto, vaciar) usando `useReducer` en lugar de varios `useState`.
10. **(Reto) Refactor anti-efecto.** Te damos un componente que usa `useEffect` para calcular un total a partir de una lista de precios y guardarlo en estado. Refactorízalo para eliminar el efecto y el estado redundante, calculando el total directamente durante el render. Justifica el cambio citando la idea de "You Might Not Need an Effect".

## Recursos y referencias

- **Documentación oficial de React** — https://react.dev (referencia principal, React 19.2).
- **Aprende React (tutorial oficial)** — https://react.dev/learn
- **Pensar en React** — https://react.dev/learn/thinking-in-react
- **No siempre necesitas un efecto** — https://react.dev/learn/you-might-not-need-an-effect
- **Reglas de los hooks** — https://react.dev/reference/rules/rules-of-hooks
- **Novedades de React 19** — https://react.dev/blog
- **React Compiler** — https://react.dev/learn/react-compiler
- **Vite** — https://vite.dev (herramienta para crear la SPA de la unidad).
- **Next.js** — https://nextjs.org (framework que veremos en la Unidad 6).